Foglio utilizzabile durante l'esame (Semestre 2)

Come nel semestre 1: smettete di cercare di ricordare la sintassi a memoria. Questo foglio vi dice **come** si scrive una cosa, non **quando** usarla né **perché**: quella è la parte che viene valutata.

Il giorno dell'esame avrete **solo questo foglio**. Del semestre 1 qui sotto c'è l'essenziale (`pytest.raises`, `parametrize`, `@patch`); il resto è quello che abbiamo usato nei lab sulle API. La parte pratica dell'esame riguarda solo API, fixture e mock: per questo qui non ci sono Playwright e GitHub Actions.

0. PROMEMORIA DAL SEMESTRE 1

```python
import pytest

def test_errore_atteso():
    with pytest.raises(ValueError):          # il test è verde solo se dentro il blocco parte ValueError
        funzione_da_testare(-10)
```

1. REQUESTS: PARLARE CON UN'API

```python
import requests

BASE_URL = "http://localhost:8000"

# GET (con parametri in query string: /api/cerca?q=yoga)
risposta = requests.get(f"{BASE_URL}/api/cerca", params={"q": "yoga"}, timeout=5)

# POST con un dizionario Python: requests lo trasforma in JSON per voi
risposta = requests.post(f"{BASE_URL}/api/risorsa", json={"campo": 1}, timeout=5)

# POST con una stringa grezza, spedita così com'è (byte per byte)
risposta = requests.post(
    f"{BASE_URL}/api/risorsa",
    data="qualsiasi testo",
    headers={"Content-Type": "application/json"},
    timeout=5,
)

# Header personalizzati (es. autenticazione)
risposta = requests.get(f"{BASE_URL}/api/privata",
                        headers={"Authorization": "Bearer abc123"}, timeout=5)
```

Leggere la risposta:

```python
risposta.status_code                 # 200, 201, 400, 401, 404, 500, 503...
risposta.headers["Content-Type"]     # "application/json"
risposta.json()                      # body JSON -> dict o list Python
risposta.json()["errore"]            # un campo del body
risposta.elapsed.total_seconds()     # quanto ci ha messo
```

Tipi Python utili nelle asserzioni:

```python
isinstance(valore, int)        # True se è un intero
isinstance(valore, str)        # True se è una stringa
isinstance(valore, list)       # True se è una lista
"chiave" in dizionario         # True se la chiave esiste
```

Validare uno schema (jsonschema):

```python
from jsonschema import validate

SCHEMA = {
    "type": "object",
    "properties": {"id": {"type": "integer"}, "nome": {"type": "string"}},
    "required": ["id", "nome"],
}
validate(instance=risposta.json(), schema=SCHEMA)   # se non è conforme -> eccezione -> test rosso
```

2. PYTEST: FIXTURE E CONFTEST.PY

Il file `tests/conftest.py` viene caricato da pytest in automatico: le fixture definite lì sono disponibili in tutti i file di test **senza import**.

```python
import pytest

@pytest.fixture                      # eseguita di nuovo per ogni test che la chiede
def nome_fixture():
    return "valore"

@pytest.fixture
def seconda(nome_fixture):           # una fixture può chiederne un'altra per nome
    return nome_fixture + "!"

@pytest.fixture(autouse=True)        # eseguita prima di ogni test, anche se nessuno la chiede
def prima_di_ogni_test():
    ...

@pytest.fixture
def con_pulizia():
    risorsa = "apri"
    yield risorsa                    # il test gira qui
    # codice dopo lo yield = teardown, eseguito alla fine del test


def test_esempio(seconda):           # il test dichiara di cosa ha bisogno: pytest fa il resto
    assert seconda == "valore!"
```

Una fixture con lo **stesso nome** definita nel file di test **sostituisce** quella del `conftest.py` per i test di quel file.

Comandi utili:

```
pytest -v                    # elenco dei test con esito
pytest -k "carrello"         # solo i test il cui nome contiene "carrello"
pytest -s                    # mostra anche i print() dei test
```

3. MOCK: UNITTEST.MOCK

Regola del semestre 1, che resta valida: si intercetta l'oggetto **dove viene usato**, non dove è definito.

```python
from unittest.mock import patch, MagicMock

@patch("modulo_di_business.oggetto_da_sostituire")
def test_con_mock(mock_oggetto):
    # Valore restituito quando il mock viene chiamato
    mock_oggetto.return_value = 42

    # Eccezione sollevata quando il mock viene chiamato
    mock_oggetto.side_effect = ValueError("guasto")

    # Con quali argomenti è stato chiamato?
    args, kwargs = mock_oggetto.call_args          # kwargs["nome_argomento"]
    mock_oggetto.assert_called_once()
    mock_oggetto.assert_called_once_with("arg1", chiave=3)   # chiamato una volta, con questi
        argomenti
```

Costruire un oggetto finto con attributi e metodi:

```
finto = MagicMock()
finto.status_code = 200                                  # attributo
finto.json.return_value = {"chiave": "valore"}           # metodo json() che restituisce un dict
finto.metodo.side_effect = RuntimeError("esplodo")       # metodo che solleva un'eccezione
```

Eccezioni della libreria requests (tutte figlie di `requests.RequestException`):

```
requests.Timeout          # il server non ha risposto entro il timeout
requests.ConnectionError  # il server non è raggiungibile
requests.HTTPError        # sollevata da risposta.raise_for_status() se lo status è >= 400
```

Matrice + eccezioni (ripasso):

```
@pytest.mark.parametrize("ingresso, atteso", [
    (1, "uno"),
    (2, "due"),
])
def test_matrice(ingresso, atteso):
    assert converti(ingresso) == atteso
```

Con `@patch` e `@pytest.mark.parametrize` insieme, gli argomenti del mock vengono **prima** di quelli della matrice: `def test_x(mock_get, ingresso, atteso):`